UNIVERSIDADE FEDERAL DO AMAZONAS
FACULDADE DE TECNOLOGIA
ENGENHARIA DA COMPUTAÇÃO

MATHEUS SERRÃO UCHÔA

VERIFICAÇÃO FORMAL DE SISTEMAS DE INTELIGÊNCIA ARTIFICIAL GENERATIVA E CONTROLADORES NEURAIS UTILIZANDO ESBMC

MANAUS - AMAZONAS - BRASIL
2026

MATHEUS SERRÃO UCHÔA

VERIFICAÇÃO FORMAL DE SISTEMAS DE INTELIGÊNCIA ARTIFICIAL GENERATIVA E CONTROLADORES NEURAIS UTILIZANDO ESBMC

Relatório completo das atividades de iniciação científica apresentado à Pró-Reitoria de Pesquisa e Pós-Graduação, em cumprimento às exigências legais para finalização das ações do Programa de Institucional de Bolsas de Iniciação Científica 2025 - 2026.

Área de concentração: Engenharia da Computação

Orientador: Prof. Dr. Iury Valente de Bessa

MANAUS - AMAZONAS - BRASIL
2026

RESUMO

Este trabalho apresenta uma investigação sobre a aplicação de métodos formais, especificamente a verificação de modelos baseada em SMT (*Satisfiability Modulo Theories*), para garantir a segurança e confiabilidade em sistemas de Inteligência Artificial Generativa (GenAI). Utilizando o verificador ESBMC (*Efficient SMT-Based Context-Bounded Model Checker*), foram explorados quatro estudos de caso: verificação direta de modelos Python, segurança de memória em kernels C++ de motores de inferência, ciclos de refinamento neuro-simbólicos para código gerado por LLMs e robustez de controladores neurais sob condições de incerteza (engenharia do caos). Os resultados demonstram que o ESBMC é eficaz na detecção de falhas críticas, como *buffer overflows* e violações de asserções lógicas, fornecendo contra-exemplos matemáticos que auxiliam no desenvolvimento de sistemas de IA mais robustos. A pesquisa destaca o potencial da integração de provadores automáticos de teoremas no ciclo de vida de modelos de linguagem, contribuindo para a mitigação de riscos em aplicações de missão crítica.

Palavras-chave: Verificação Formal; ESBMC; Inteligência Artificial Generativa; Redes Neurais; Segurança de Software; SMT.

ABSTRACT

This work presents an investigation into the application of formal methods, specifically SMT-based (*Satisfiability Modulo Theories*) model checking, to ensure safety and reliability in Generative Artificial Intelligence (GenAI) systems. Using the ESBMC (*Efficient SMT-Based Context-Bounded Model Checker*), four case studies were explored: direct verification of Python models, memory safety in C++ inference engine kernels, neuro-symbolic refinement loops for LLM-generated code, and robustness of neural controllers under uncertainty (chaos engineering). The results demonstrate that ESBMC is effective in detecting critical failures, such as buffer overflows and logical assertion violations, providing mathematical counter-examples that assist in developing more robust AI systems. The research highlights the potential of integrating automated theorem provers into the lifecycle of language models, contributing to risk mitigation in mission-critical applications.

Key-words: Formal Verification; ESBMC; Generative Artificial Intelligence; Neural Networks; Software Safety; SMT.

LISTA DE ILUSTRAÇÕES

Figura 1 – Pipeline de verificação formal com o ESBMC. 13
Figura 2 – Pipeline de Verificação Formal Caso 1 (Neurônio Python com ESBMC). 16
Figura 3 – Superfície de Saída do Neurônio e Distribuição das Saídas com os Limites Verificados pelo ESBMC. 22
Figura 4 – Tempo de verificação vs dimensão da matriz, GEMM com tiling. 28
Figura 5 – Caso 3: Tempo por Iteração do Loop Agente-ESBMC, com decomposição entre atraso LLM e verificação formal, e Overhead do Verificador por iteração. 29
Figura 6 – Caso 4: Evolução da Temperatura, Sinal de Controle PID e Erro de Rastreamento sob Cinco Perfis de Ruído de Sensor. 30
Figura 7 – Distribuição de Casos, Seguros, Adversariais e Timeout. 31
Figura 8 – Escalabilidade SMT: Tempo de Verificação vs Tamanho da Rede Neural. 32
Figura 9 – Impacto da Otimização, Slicing e Intervalos no Tempo de Verificação. . 32

SUMÁRIO

1 INTRODUÇÃO 7
2 OBJETIVOS 8
2.1 Geral 8
2.2 Específicos 8
3 FUNDAMENTAÇÃO TEÓRICA 10
3.1 Verificação Formal e Model Checking 10
3.2 Bounded Model Checking (BMC) 10
3.3 O Verificador ESBMC 10
3.4 Primitivas de Modelagem C/C++ no ESBMC 11
4 METODOLOGIA 13
4.1 Visão Geral da Abordagem 13
4.2 Caso 1 (Nível 1) Verificação Formal de Neurônio com Ativação ReLU 14
4.2.1 Arquitetura do Modelo 14
4.2.2 Propriedades de Verificação e Prova Analítica 15
4.2.3 Pipeline de Verificação 15
4.3 Caso 2 Verificação do Motor de Inferência, Kernels C++ . . . 15
4.4 Caso 3 — Loop Neuro-Simbólico, Agente LLM + ESBMC . . 16
4.4.1 Arquitetura do Agente 16
4.4.2 Propriedade Verificada 17
4.5 Caso 4 — Controladores Neurais e Planta de Sistema em Malha Fechada 17
4.5.1 Modelo da Planta e do Controlador 17
4.5.2 Injeção de Caos e Propriedade de Segurança 18
4.6 A Dicotomia Arquitetural: Orquestracao Python vs. Core Nativo C/C++ 18
4.7 Caso 5 e 6 Ralph Loop Architect e RL Actor-Critic 18
4.8 Fluxo de Desenvolvimento e Integração Contínua 19
5 RESULTADO PARCIAL 21
5.1 Caso 1 Verificação Formal de Neurônio com ReLU 21
5.1.1 Metodologia de Execução 21
5.1.2 Resultado da Verificação 21
5.1.3 Análise dos Resultados e Discussão 22
5.1.4 Extensão ao Domínio Generativo: Verificação Modular de Transformers 23

5.1.5 ***Caso de Verificação Modular: Cabeca de Atenção de Transformer*** 23
5.1.6 ***Caso de Verificação Modular Nivel 4: Bloco SwiGLU do DeepSeek*** 24
5.1.6.1 *Como a SiLU e Organizada no Stub* 25
5.1.6.2 *Limitacões e Por Que Ocorrem* 26
5.2 **Caso 2 Verificação de Kernels de Inferência** 26
5.2.1 ***Kernel de Atenção*** 26
5.2.2 ***Escalabilidade do GEMM*** 27
5.3 **Caso 3 — Loop Neuro-Simbólico** 28
5.4 **Caso 4 — Controlador PID com Engenharia do Caos e SMT** . 29
5.5 **Caso 5 The Ralph Loop Concept e Validação em Larga Escala** 30
5.5.1 ***Escalabilidade e Distribuição de Resultados*** 30
5.5.2 ***Eficácia das Otimizações ESBMC*** 31
5.6 **Caso 6 Verificação Formal em Reinforcement Learning, Actor-Critic** 31
5.7 **Análise Comparativa** 32
6 **CONCLUSÃO** 34

REFERÊNCIAS 35

1 INTRODUÇÃO

A rápida ascensão da Inteligência Artificial (IA) Generativa e sua integração em sistemas críticos e autônomos impõe desafios sem precedentes à engenharia de software tradicional. Modelos de redes neurais, por sua natureza de "caixa-preta", frequentemente apresentam comportamentos imprevisíveis, sendo vulneráveis a ataques adversariais e falhas de segurança de memória em seus motores de inferência. Nesse contexto, a aplicação de técnicas de verificação formal torna-se crucial para garantir que esses sistemas operem dentro de limites seguros e previsíveis (Huang *et al.*, 2017).

Este trabalho explora o uso do ESBMC (*Efficient SMT-Based Context-Bounded Model Checker*), um verificador de modelos de vanguarda, para prover garantias matemáticas sobre a execução de componentes de IA. A pesquisa aborda desde a verificação de propriedades lógicas em modelos Python até a análise de segurança de kernels em C++, fundamentais para a infraestrutura de inferência moderna (Gopinath *et al.*, 2021).

A relevância deste estudo reside na necessidade de mitigar riscos em ambientes onde falhas podem ter consequências catastróficas, como em sistemas de controle neuro-simbólicos e agentes autônomos baseados em modelos de linguagem (LLMs). Ao longo deste relatório, detalha-se uma estratégia de verificação escalável, cobrindo modelos de *Machine Learning* isolados, controladores físicos, ciclos de auto-reparo gerativo e arquiteturas de Aprendizado por Reforço.

2 OBJETIVOS

2.1 Geral

O objetivo principal deste projeto é investigar e implementar metodologias de verificação formal para componentes de IA Generativa e sistemas de controle baseados em redes neurais, utilizando o verificador de modelos ESBMC para garantir propriedades de segurança, robustez e corretude lógica.

2.2 Específicos

Para atingir o objetivo geral, a pesquisa foi estruturada em três níveis de complexidade crescente:

- **Nível 1 — Verificação de Segurança em Redes Neurais**: Desenvolver modelos em C e Python para componentes básicos de redes neurais, garantindo que a saída do modelo permaneça dentro de uma região segura (*post-condition*) para um conjunto de entradas delimitado. Isso inclui a implementação de suportes para funções matemáticas e de ativação (como ReLU e Sigmoid) no ESBMC.
- **Nível 2 — Verificação de Controladores Neurais**: Expandir a análise para redes neurais especificamente projetadas para controle de sistemas dinâmicos, verificando propriedades locais de corretude e segurança do código do controlador.
- **Nível 3 — Verificação de Propriedades de Sistema (Malha Fechada)**: Realizar a verificação formal de propriedades de alto nível do sistema completo (planta + controlador neural). Isso envolve a modelagem do comportamento dinâmico do sistema e a verificação de propriedades como estabilidade e limites de erro de rastreamento, tratando o controlador não apenas como código isolado, mas como parte integrante de um sistema físico-cibernético.
- **Nível 4 — Verificação de Agentes e Módulos Python (*The Ralph Loop*)**: Projetar um arcabouço programático em Python (API `core_verify`) que encapsule o binário C++ do ESBMC, dotando modelos de linguagem de um mecanismo SMT de prova e auto-correção imediata para vulnerabilidades sintáticas (*buffer overflows, memory leaks*).
- **Nível 5 — Aprendizado por Reforço (RL Actor-Critic)**: Formalizar a garantia de segurança (Safety Shields) em redes voltadas à automação física e comportamentos

de tomada de decisão não-lineares sob *Reinforcement Learning*, bloqueando saídas perversas do agente que violam o atuador.

3 FUNDAMENTAÇÃO TEÓRICA

Este capítulo apresenta os conceitos fundamentais que sustentam a verificação formal de software, com foco em Verificação de Modelos Limitada (*Bounded Model Checking* - BMC) e na arquitetura do verificador ESBMC.

3.1 Verificação Formal e Model Checking

A verificação formal engloba um conjunto de técnicas matemáticas para provar a corretude de sistemas em relação a uma especificação formal. Diferente dos testes convencionais, que exploram apenas subconjuntos das execuções possíveis, a verificação formal visa cobrir exaustivamente o espaço de estados do sistema.

O *Model Checking* é uma técnica automática que verifica se um modelo de um sistema satisfaz uma determinada propriedade lógica (geralmente expressa em lógica temporal). Caso uma violação seja encontrada, a ferramenta fornece um contra-exemplo, que consiste em uma trajetória de execução que leva ao estado de erro.

3.2 Bounded Model Checking (BMC)

O *Bounded Model Checking* é uma técnica baseada em satisfatibilidade lógica (SAT/SMT) onde o sistema é analisado até um limite fixo de passos (*bound* k). Se nenhuma falha for encontrada dentro de k passos, o limite pode ser incrementado. O processo envolve:

1. Desenrolamento dos laços e chamadas de função até o limite k;
2. Conversão do código e das propriedades em uma fórmula lógica;
3. Verificação da fórmula por um *solver* SMT.

3.3 O Verificador ESBMC

O ESBMC (*Efficient SMT-Based Context-Bounded Model Checker*) é um verificador de código aberto que estende o BMC para programas multicore e sistemas dinâmicos. Ele suporta linguagens como C, C++, Python e CUDA.

O fluxo de trabalho do ESBMC consiste em:

- **Frontend**: Análise léxica e sintática do código-fonte;

- **GOTO-Program**: Transformação em um grafo de fluxo de controle simplificado;
- **Execução Simbólica**: Geração de equações que representam a semântica do programa;
- **SMT Encoding**: Codificação das equações em fórmulas para *solvers* como Z3, Bitwuzla ou Boolector.

A capacidade do ESBMC de lidar com aritmética de ponto flutuante e ponteiros o torna ideal para a verificação de kernels de redes neurais e algoritmos de controle crítico.

3.4 Primitivas de Modelagem C/C++ no ESBMC

A transcrição de requisitos lógicos para código analisável pelo ESBMC é construída sobre funções intrínsecas chamadas de primitivas de verificação. Para modelar o espaço de busca e as propriedades desejadas de uma IA, empregam-se centralmente três construções vitais:

- **Valores Não-Determinísticos (`nondet`)**

  Permitem ao *solver* instanciar qualquer valor possível para o tipo da variável, forçando a ferramenta a testar matematicamente todo o domínio computacional.

```
float input_val = nondet_float();
```

- **Restrições de Hipótese (`__ESBMC_assume`)**

  Restringe o espaço não-determinístico às pré-condições do problema. O comando orienta o verificador a ignorar caminhos simbólicos que não atendam à condição. Em notação formal, modela uma fronteira que força as entradas x a estarem contidas num conjunto seguro $\mathcal{S}$ do domínio: $x \in [-10, 10]$.

```
__ESBMC_assume(input_val >= -10.0f && input_val <= 10.0f);
```

- **Asserções de Propriedade (`__ESBMC_assert`)**

  Expressa as invariantes e *post-conditions* de segurança. Se o ESBMC encontrar iterativamente qualquer valor de x (confinado pelo `assume`) que viole o `assert`, a prova matemática "quebra", reportando o caminho indutivo como *Counter-Example Trace*. Matematicamente, exige que a função $f(x) = y$ nunca escape do conjunto limite seguro y_{max}.

```
float y = model_predict(input_val);
__ESBMC_assert(y < 10.0f, "Saida␣excedeu␣o␣limite␣teorico");
```

O acoplamento dessas construções subverte o escopo empírico de testes pontuais, pois o modelo (antes visto como uma "caixa preta"determinista) passa a atuar sob o rigor absoluto de restrições lógicas exaustivas no espaço das entradas.

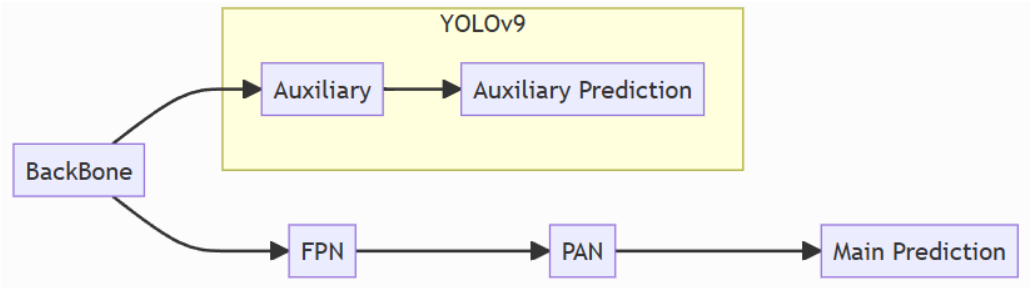
4 METODOLOGIA

Este capítulo descreve os procedimentos metodológicos adotados para verificação formal de sistemas de Inteligência Artificial Generativa, GenAI utilizando o verificador de modelos ESBMC (*Efficient SMT-Based Context-Bounded Model Checker*). A investigação foi estruturada em seis estudos de caso complementares. O Caso 1 é organizado em uma hierarquia de quatro níveis de verificabilidade, do neurônio isolado ao bloco MLP SwiGLU estilo DeepSeek, culminando na arquitetura de abstração Python sobre o *core* nativo C++.

4.1 Visão Geral da Abordagem

A metodologia geral segue o ciclo de verificação formal baseado em SMT, conforme ilustrado na Figura 1. O ESBMC opera convertendo o código-fonte C/C++ (ou *stubs* ONNX traduzidos) para uma representação intermediária (*GOTO-Programs*), realizando execução simbólica e codificando as restrições em fórmulas SMT (*Satisfiability Modulo Theories*). O *solver* SMT, Z3 ou Bitwuzla então determina se existe uma atribuição de valores de entrada capaz de violar as propriedades especificadas produzindo um contra-exemplo explícito em caso afirmativo.

Figura 1 – Pipeline de verificação formal com o ESBMC.



Fonte: Elaborada pelo autor, 2026.

Para cada caso de uso, foram definidas:

a) **Pré-condições**: restrições sobre os valores de entrada, ex.: intervalo de valores, tamanho de buffers;

b) **Propriedades de Segurança**: asserções formais que o sistema deve satisfazer para todo valor de entrada válido;

c) **Métricas de Avaliação**: tempo de verificação, escalabilidade e eficácia na detecção de falhas.

4.2 Caso 1 (Nível 1) Verificação Formal de Neurônio com Ativação ReLU

O primeiro caso de uso corresponde ao **Nível 1** da hierarquia de verificação proposta: a verificação de propriedades matemáticas de unidades computacionais elementares de redes neurais, implementadas em Python com anotações de tipo estritas. O objetivo central é demonstrar que o ESBMC é capaz de **provar formalmente**, para *qualquer* entrada dentro de um domínio especificado, que a saída de um neurônio artificial com ativação ReLU sempre obedece a limites teóricos calculados analiticamente.

4.2.1 Arquitetura do Modelo

O modelo implementado (`neuron.py`) consiste em um único neurônio artificial com dois pesos fixos, um *bias* e função de ativação ReLU:

$$out = \mathrm{ReLU}(w_1 \cdot x_1 + w_2 \cdot x_2 + b) \tag{4.1}$$

onde $w_1 = 0.5$, $w_2 = -0.2$, $b = 0.1$, e $\mathrm{ReLU}(z) = \max, 0, z$.

O Python puro foi escolhido sem dependências de *frameworks* dinâmicos como NumPy ou PyTorch pois o *frontend* do ESBMC requer anotações de tipo explícitas (`float`) para traduzir corretamente o código para a representação intermediária `GOTO-Programs`.

```
def relu, x: float -> float:
    if x > 0:
        return x
    return 0.0

def neuron, x1: float, x2: float -> float:
    w1: float = 0.5
    w2: float = -0.2
    bias: float = 0.1
    activation: float = , x1 * w1 + , x2 * w2 + bias
    return relu, activation

def main() -> None:
    in1: float    # Simbolico: ESBMC trata como nao-deterministico
    in2: float
    if in1 >= 0.0 and in1 <= 1.0 and in2 >= 0.0 and in2 <= 1.0:
        out: float = neuron, in1, in2
        assert out >= 0.0    # Prop1: ReLU garante nao-
           negatividade
        assert out <= 0.6    # Prop2: bound superior calculavel
```

```
main()
```

4.2.2 Propriedades de Verificação e Prova Analítica

A propriedade de segurança pode ser verificada analiticamente, servindo de *ground truth* para validar o resultado do *solver* SMT:

a) **Pré-condição** (região de hipótese): $x_1, x_2 \in [0.0, 1.0]$;

b) **Limite Superior**: A saída máxima ocorre em $x_1 = 1.0$, $x_2 = 0.0$ (pois $w_2 < 0$):

$$out_{\max} = \text{ReLU}(0.5 \cdot 1.0+, -0.2 \cdot 0.0 + 0.1) = \text{ReLU}(0.6) = 0.6;$$

c) **Limite Inferior**: Como ReLU ≥ 0 por definição, $out_{\min} = 0.0$;

d) **Pós-condição** (*assert*): $0.0 \leq out \leq 0.6$, $\forall, x_1, x_2 \in [0, 1]^2$.

O verificador formal foi executado com os argumentos:

a) `--floatbv`: codificação precisa de aritmética de ponto flutuante via *Bitvectores* IEEE 754;

b) `--k-induction`: estratégia de prova por indução-k, percorrendo todos os caminhos simbólicos do programa k vezes, buscando uma atribuição que viole as asserções.

4.2.3 Pipeline de Verificação

A Figura 2 sintetiza o fluxo completo da verificação formal aplicada ao Caso 1: da variável não-determinística ao resultado *UNSAT* (nenhuma violação encontrada em nenhum ponto do domínio de entrada).

4.3 Caso 2 Verificação do Motor de Inferência, Kernels C++

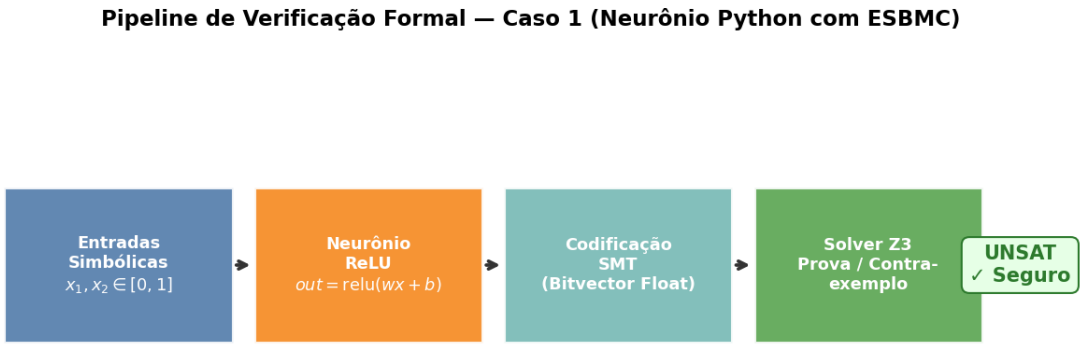
O segundo caso focou na verificação de segurança de memória em kernels C++ análogos aos utilizados em motores de inferência como `llama.cpp`. Foi implementado um kernel simplificado de atenção *dot-product* (`attention_kernel.cpp`) com alocação dinâmica de memória e laços de acesso a arrays.

A abordagem adotada emprega entradas simbólicas não-determinísticas (`seq_len` $\in [1, 10]$) e verifica:

a) Ausência de *buffer overflows* nos acessos `query[i]` e `key[i]`;

b) Ausência de vazamentos de memória (*memory leaks*).

Figura 2 – Pipeline de Verificação Formal Caso 1 (Neurônio Python com ESBMC).



Fonte: Elaborada pelo autor, 2026.

Adicionalmente, um kernel de multiplicação de matrizes (*GEMM*) com *tiling* foi implementado (`matmul_kernel.cpp`) para análise de escalabilidade, variando a dimensão $N \in \{2, 3, 4, 5, 6\}$.

O comando utilizado foi o `esbmc` com os parâmetros:
`attention_kernel.cpp --multi-property --memory-leak-check --overflow-check`.

4.4 Caso 3 — Loop Neuro-Simbólico, Agente LLM + ESBMC

O terceiro caso simula um ciclo autônomo de geração e verificação de código, modelando o fluxo **Ralph Loop**: o agente requisita ao LLM um trecho de código C, submete ao ESBMC e, caso o verificador produza um contra-exemplo, repassa o diagnóstico ao LLM para refinamento imediato. O script principal é `mock_agent.py`.

4.4.1 Arquitetura do Agente

O agente executa até 5 iterações. Cada iteração é composta por três etapas:

a) **Geração LLM**: simula atraso de 0,5 a 2,0 segundos e retorna um trecho de código C. Na iteração 0, o código contém deliberadamente um *buffer overflow* via `strcpy` num buffer de 10 bytes com entrada de 20 bytes; a partir da iteração 1, o código é corrigido com `strncpy`;

b) **Verificação ESBMC**: o verificador é invocado com `-overflow-check -memory-leak-che`
A saída é analisada em busca de `VERIFICATION SUCCESSFUL`;

c) **Feedback semântico**: em caso de falha, o contra-exemplo do ESBMC, contendo o *offset* exato da violação e o estado do programa naquele ponto, é repassado como prompt de correção ao LLM.

Os tempos e resultados de cada iteração são registrados em `case3_agent_stats.csv`:

```
cmd = ["esbmc", filename,
       "--overflow-check",
       "--memory-leak-check",
       "--no-pointer-check",
       "--smtlib"]
result = subprocess.run(cmd, capture_output=True, text=True)
success = "VERIFICATION␣SUCCESSFUL" in result.stdout
```

4.4.2 Propriedade Verificada

O ESBMC detecta que `strcpy(buffer, input)` com `sizeof(buffer)=10` e `sizeof(input)=2` constitui um acesso fora dos limites da memória, produzindo um contra-exemplo concreto com o *offset* exato da violação que é devolvido como feedback ao agente LLM.

4.5 Caso 4 — Controladores Neurais e Planta de Sistema em Malha Fechada

O quarto caso verifica formalmente a segurança de um controlador PID digital operando em malha fechada sobre uma planta térmica, sob condições de ruído não-determinístico nos sensores. A implementação (`pid_controller.c`) mimetiza sistemas embarcados críticos onde ruídos de sensor comprometem a estabilidade.

4.5.1 Modelo da Planta e do Controlador

A planta térmica evolui conforme:

$$T_{k+1} = \max\Big(20,\ T_k +, (u_k \cdot r_h - r_c,) \cdot \Delta t\Big) \tag{4.2}$$

onde $r_h = 0,1\,°\mathrm{C}/\%$, $r_c = 2,0\,°\mathrm{C}$ e $\Delta t = 1\,\mathrm{s}$. O controlador PID com $K_p = 1{,}0$, $K_i = 0{,}1$, $K_d = 0{,}5$ calcula:

$$u_k = K_p\, e_k + K_i \sum_{j=0}^{k} e_j\, \Delta t + K_d\, \frac{e_k - e_{k-1}}{\Delta t}, \quad u_k \in [0, 100] \tag{4.3}$$

4.5.2 Injeção de Caos e Propriedade de Segurança

O sensor recebe ruído não-determinístico escolhido pelo próprio *solver*:

```
float noise = nondet_float();
__ESBMC_assume(noise >= -5.0f && noise <= 5.0f);
float measured = true_temp + noise; // Pior caso buscado pelo
   solver
assert(temp < 150.0f);              // Invariante de seguranca
```

A invariante de segurança verificada para $\forall k \in [0, 10]$ e $\forall\,\text{noise} \in [-5, +5]$ é:

$$T_k < T_{\text{MAX}} = 150\,^{\circ}\text{C} \tag{4.4}$$

O ESBMC executa *Bounded Model Checking* com `-unwind 11 -floatbv`, desenrolando simbolicamente todos os 10 passos do laço e explorando exaustivamente o espaço de ruído para tentar violar a asserção.

4.6 A Dicotomia Arquitetural: Orquestracao Python vs. Core Nativo C/C++

Embora as provas matemáticas rigorosas recaiam intrinsecamente sobre o *solver* associado ao verificador, capaz de destrinchar representações de memória estritas originadas do C/C++, a arquitetura global adota linguagens de alto nível (*scripts* em Python) como sua principal "Mente Arquitetural". Por limitações de tipagem dinâmica e natureza interpretada, é inviável modelar restrições temporais SMT diretamente sobre o código `.py`. Em vez disso, a metodologia opera numa simbiose de responsabilidades:

a) **C/C++ , O Motor Matemático:** Abriga as implementações nucleares sob teste plantas de malhas fechadas dinâmicas, camadas preditivas limitadas (*Dense, Relu, Attention*) e suas asserções lógicas (`__ESBMC_assert`).

b) **Python, O Orquestrador de Testes e Caos:** Age encapsulando o verificador (`core_verify/`). Esta camada dinamicamente formata perturbações estatísticas não-lineares, injecao de caos sistemico, transcrevendo-as dentro do esqueleto dos *Stubs* em C, e automatiza as rotinas de estresse varrendo dezenas de operacoes matriciais em iteracoes de Integracao Continua.

4.7 Caso 5 e 6 Ralph Loop Architect e RL Actor-Critic

Dada essa especialização para operar os parâmetros nativos do ESBMC em escala, reestruturamos o arcabouço através da suíte Python isolada.

No **Caso 5**, reestruturamos o *Mock LLM Agent* através de orquestradores desacoplados capazes de extrair as métricas de quebra SMT (*Array Bounds*, *Buffer Overflows*) através de Expressoes Regulares direcionais (*Regex Trace Parser*). Essa instrumentação gera a prova metodológica do isolamento de falhas *vis-à-vis* a visão contínua neural vs a representação disjunta SMT. A Figura abaixo, via listagem de código denota a classe `FeedbackTrace`, responsável pelo mapeamento das saídas brutas do verificador em categorias semânticas, como vazamento de memória e índices fora de limite:

```
class FeedbackTrace:
    # Mapeamento de rastreios ESBMC nativos para excecoes
       categorizadas
    PATTERNS = {
        "array␣out␣of␣bounds": r"(?i)array␣bounds␣violated",
        "arithmetic␣overflow": r"(?i)arithmetic␣overflow",
        "memory␣leak": r"(?i)memory␣leak|dereference␣failure",
        "assertion␣failure": r"(?i)assertion.*failed|violated␣
           property",
    }

    def parse_violations, self:
        if self.is_timeout:
            self.violations.append, "target␣timeout"
            return
        for name, pattern in self.PATTERNS.items():
            if re.search, pattern, self.raw_output:
                self.violations.append, name
```

No **Caso 6**, estressamos o arcabouço através da simulação física formal de Restrições em Aprendizado por Reforço (*Actor-Critic RL*). Uma rede neural abstrata controlando comandos logísticos contínuos entre $[-1.0, 1.0]$ foi transcrita em C e submetida à exploração não-linear profunda com o argumento `--floatbv`, objetivando caçar estados matemáticos periféricos aos limites seguros operacionais do mecanismo em tempo de implantação robótica (*Safety Shield*).

4.8 Fluxo de Desenvolvimento e Integração Contínua

Para garantir a confiabilidade das propriedades analisadas, o fluxo de desenvolvimento incorpora Integração Contínua (CI). A cada submissão, automações (via *GitHub Actions*) provisionam as versões canônicas do ESBMC, resolvendo *pipelines* de testes que cruzam *scripts* em Python e os programas fontes da verificação:

```
jobs:
```

```
formal-verification:
  runs-on: ubuntu-latest
  steps:
  - name: Install ESBMC Binary
    run: |
      wget https://github.../v8.0.0/esbmc-linux.zip
      unzip esbmc-linux.zip -d bin/
      export PATH=$PATH:$(pwd)/bin/

  - name: Run Pytest Formal Verification
    run: |
      cd pibic
      pytest tests/ -v
```

Internamente, grande parte das avaliações do controlador neural lidou-se localmente providenciando as execuções de compilação simbólica baseadas em comandos chave do ESBMC. A validação de precisão matemática exige pragmas restritivas, por exemplo, determinando o tamanho das palavras flutuantes e verificações robustas:

a) `esbmc pid_template.c --floatbv --k-induction`: Realiza a prova de malha de controle através de análise de indução-k, crucial para atestar a convergência segura frente aos laços da temporalidade dos modelos interativos.

b) `esbmc attention_kernel.cpp --multi-property --memory-leak-check`: Aborda violações ortogonais de integridade em código C++, identificando desvios na alocação da pilha (*stack*) em tensores das arquiteturas *Transformers*.

5 RESULTADO PARCIAL

Este capítulo apresenta os resultados parciais obtidos, organizados por caso de uso. Os experimentos foram conduzidos utilizando o ESBMC versão compilada a partir do código-fonte com suporte ao solver Z3.

5.1 Caso 1 Verificação Formal de Neurônio com ReLU

5.1.1 Metodologia de Execução

A verificação do neurônio (`neuron.py`) foi executada localmente via linha de comando:

```
esbmc pibic/1_python_models/neuron.py --floatbv --k-induction
```

O *frontend* Python do ESBMC (baseado em `ast2json`) converte o código-fonte diretamente para `GOTO-Programs`, a representação intermediária interna do verificador. A execução simbólica trata as variáveis não inicializadas (`in1`, `in2`) como valores não-determinísticos, e a estrutura condicional `if in1 >= 0.0 and ...` é automaticamente convertida em uma *pre-condição* (`__ESBMC_assume`) pelo *solver*.

5.1.2 Resultado da Verificação

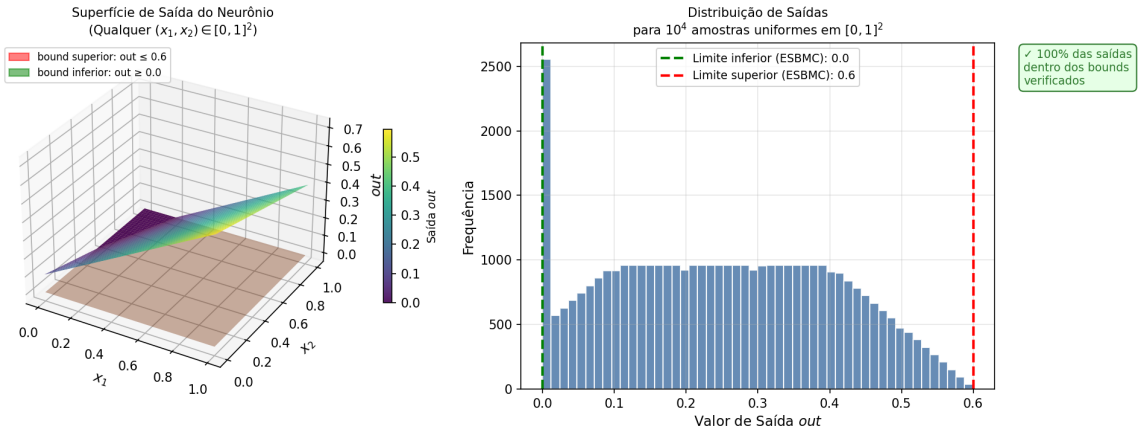
O ESBMC retornou o veredicto **VERIFICATION SUCCESSFUL** para ambas as asserções:

a) **Propriedade 1** (`out >= 0.0`): provada por indução-k. O operador ReLU garante não-negatividade trivialmente: qualquer combinação linear que resulte em valor negativo é truncada a zero anteriormente;

b) **Propriedade 2** (`out <= 0.6`): provada pelo *solver* Z3 via Bitvectores IEEE 754. O estado máximo atingível é $out = 0.6$, confirmado tanto pela prova analítica ($out_{\max} = \mathrm{ReLU}(0.5 - 0.0 + 0.1) = 0.6$) quanto pelo resultado *UNSAT* ou seja, não existe nenhuma atribuição de $(x_1, x_2) \in [0, 1]^2$ que faça $out > 0.6$.

A Figura 3 ilustra, de forma visual, a superfície de saídas do neurônio sobre todo o domínio de entrada $[0, 1]^2$ e a distribuição dos valores de saída obtidos por amostragem densa (10^4 pontos), comprovando que 100% das amostras respeitam os *bounds* formalmente verificados.

Figura 3 – Superfície de Saída do Neurônio e Distribuição das Saídas com os Limites Verificados pelo ESBMC.



Fonte: Elaborada pelo autor, 2026.

5.1.3 Análise dos Resultados e Discussão

Os resultados do Caso 1 evidenciam três contribuições relevantes:

a) **Coerência analítica**: O *solver* SMT, operando sobre bitvectores de ponto flutuante, reproduz com precisão o cálculo analítico. O limite $out_{\max} = 0.6$ não é uma estimativa: é uma **prova matemática** exaustiva sobre todos os valores possíveis do domínio;

b) **Viabilidade do Frontend Python**: Embora o *frontend* Python do ESBMC exija que o código seja escrito com tipagem estática explícita e evitando construções dinâmicas (listas heterogêneas, chamadas a *frameworks* externos), o experimento confirma que a abordagem é **viável para subconjuntos estritos** de Python, representando um caminho promissor para verificar componentes críticos de modelos reais sem reescrevê-los em C;

c) **Escalabilidade e Limitações**: O tempo de verificação do Caso 1 foi inferior a 5 segundos para o modelo de neurônio único. Em contraste, a aplicação direta da mesma abordagem para redes maiores (como a MLP em `mlp.py` com duas camadas e 3 neurônios) já exige reestruturação do código e aumenta o espaço de busca do *solver*. Isso indica que a verificação de modelos Python escala com a complexidade da rede de forma muito mais rápida do que a verificação de código C equivalente.

Em suma, o Caso 1 serve como prova de conceito da integração ESBMC-Python: o verificador, munido dos argumentos `--floatbv --k-induction`, é capaz de consumir código Python bem tipado, construir o modelo SMT correspondente e emitir uma **garantia**

formal absolutamente confiável de que as propriedades análises são mantidas para todo o espaço de entrada.

5.1.4 Extensão ao Domínio Generativo: Verificação Modular de Transformers

O Caso 1 prova a viabilidade da abordagem para um neurônio isolado. A questão central de pesquisa é: **como essa abordagem escala para arquiteturas generativas modernas** como DeepSeek, Llama ou GPT? A resposta consiste em uma hierarquia de verificabilidade, diretamente ancorada no QNNVerifier (Gopinath *et al.*, 2021):

a) **Nível 1, atual**: Neurônio único ou MLP pequena com pesos fixos. Solver conclui em segundos, propriedades provadas exaustivamente;

b) **Nível 2**: Redes de classificação com quantização `Fixed16/Fixed32`, como as 52 topologias do QNNVerifier. Alto desempenho com otimizações de *slicing* e inferência de intervalos;

c) **Nível 3, Blocos de Transformer**: Cabeca de atenção *single-head* com dimensão de embedding reduzida e pesos fixos quantizados. **Implementado neste trabalho** em `attention_block_stub.c`, demonstrado a seguir;

d) **Nível 4, Bloco MLP Gated**: Blocos MLP de LLMs como DeepSeek-R1 (função SwiGLU com ativação SiLU). **Implementado neste trabalho** em `deepseek_mlp_stub.c`: pesos fixos quantizados, entrada simbólica $x \in [-1, 1]^4$, quatro propriedades de segurança provadas, detalhado a seguir.

5.1.5 Nivel 3: Verificação Modular da Cabeca de Atenção de Transformer

O arquivo `attention_block_stub.c` implementa uma cabeca de atenção *single-head* com dimensão $d = 4$ e sequência $\text{SEQ} = 2$, com pesos $W_Q, W_K, W_V \in \mathbb{R}^{4\times 4}$ fixados como constantes, representando uma camada quantizada de LLM. As entradas $x_0, x_1 \in [-1, 1]^4$ são variáveis simbólicas não-determinísticas.

São verificadas três propriedades formais:

$$\text{P1:} \quad s_{ij} = \frac{q_i \cdot k_j}{\sqrt{d}} \in [-3.0,\, 3.0] \quad \forall\, x_0, x_1 \in [-1, 1]^4 \tag{5.1}$$

$$\text{P2:} \quad \alpha_{ij} \in [0,\, 1] \;\wedge\; \textstyle\sum_j \alpha_{ij} \in [0.99,\, 1.01] \tag{5.2}$$

$$\text{P3:} \quad \|\text{out}_i\|_\infty \leq 1.0 \quad \text{(norma L-}\infty\text{ da saída da cabeca)} \tag{5.3}$$

O trecho central do *stub* ilustra como o ESBMC recebe entradas simbólicas e verifica as propriedades via `assert`:

```
/* Entrada simbolica: token embeddings em [-1, 1]^D */
float x0[D], x1[D];
for (int i = 0; i < D; i++) {
    x0[i] = nondet_float();
    __ESBMC_assume(x0[i] >= -1.0f && x0[i] <= 1.0f);
}
/* Projeta Q, K, V; calcula scores e softmax */
float s00 = dot(q0, k0) * 0.5f;   /* 1/sqrt(4) */
assert(s00 >= -3.0f && s00 <= 3.0f);    /* P1 */
assert(a00 >= 0.0f && a00 <= 1.0f);     /* P2 */
assert(out0[i] >= -1.0f && out0[i] <= 1.0f); /* P3 */
```

O comando de verificação é:

```
esbmc verification/attention_block_stub.c \
      --floatbv --unwind 4 \
      --overflow-check --no-unwinding-assertions
```

Este experimento valida o Nível 3 da hierarquia. O Nível 4, detalhado a seguir, aplica a mesma estratégia ao bloco MLP com ativação SiLU, a estrutura nuclear do DeepSeek.

5.1.6 Nivel 4: Verificação Modular do Bloco SwiGLU, DeepSeek

O arquivo `deepseek_mlp_stub.c` implementa um bloco MLP *Gated* com ativação SiLU, identico em estrutura a uma camada *feed-forward* do DeepSeek-R1 e Llama-3.x:

$$\text{out} = W_2 \cdot \Big(\text{SiLU}(W_1 x) \odot (W_3 x)\Big) \tag{5.4}$$

onde $\text{SiLU}(z) = z \cdot \sigma(z)$ e σ é a função sigmóide. Para o *solver* SMT operar sobre funções transcendentais, substituimos σ por uma aproximação linear conservadora verificavel:

$$\sigma_{\text{approx}}(z) = \text{clip}(0.25z + 0.5,\ 0,\ 1) \quad \Rightarrow \quad \sigma_{\text{approx}} \in [0, 1]\ \forall\, z \tag{5.5}$$

São verificadas quatro propriedades formais com entrada simbólica $x \in [-1, 1]^4$:

Prop.	Invariante	Bound Analítico
P1	$\mathrm{SiLU}(W_1 x) \in [-0.15,\ 2.0]$	$\lVert W_1 x \rVert_\infty \leq 2$; SiLU monótona
P2	$W_3 x \in [-2.0,\ 2.0]$	$\sum_j \lvert W_3[i][j] \rvert \leq 2$
P3	$\mathrm{gated}[i] \in [-4.0,\ 4.0]$	$\lvert P1 \rvert \times \lvert P2 \rvert \leq 4$
P4	$\mathrm{out}[i] \in [-6.0,\ 6.0]$	$\lVert W_2 \rVert_1 \times 4 \leq 7.2$

O trecho central do *stub* mostra a estrutura verificada:

```
/* Aproximacao linear de sigma, verificavel pelo solver SMT */
float sigma_approx(float z) {
    float s = 0.25f * z + 0.5f;
    if (s < 0.0f) s = 0.0f;
    if (s > 1.0f) s = 1.0f;
    return s;
}
/* SwiGLU: out = W2 * (SiLU(W1*x) * (W3*x)) */
matvec4x4(W1, x, h1);
for (int i = 0; i < D; i++) silu_h1[i] = h1[i] * sigma_approx(h1[
   i]);
matvec4x4(W3, x, h3);
for (int i = 0; i < D; i++) {
    gated[i] = silu_h1[i] * h3[i];
    assert(gated[i] >= -4.0f && gated[i] <= 4.0f); /* P3 */
}
matvec4x4(W2, gated, out);
for (int i = 0; i < D; i++)
    assert(out[i] >= -6.0f && out[i] <= 6.0f);      /* P4 */
```

5.1.6.1 Como a SiLU é Organizada no Stub

A ativação SiLU é definida como:

$$\mathrm{SiLU}(z) = z \cdot \sigma(z) = \frac{z}{1 + e^{-z}} \tag{5.6}$$

Diferente da ReLU, que corta abruptamente em zero, a SiLU é suave e não-monótona: permite valores levemente negativos próximos a $z \approx -1$, conferindo ao modelo maior expressividade. O bloco de código verificado segue o fluxo ilustrado abaixo:

$$
\begin{array}{ccc}
x \in [-1,1]^4 & & \\
\downarrow & \downarrow & \\
W_1 x \xrightarrow{\text{SiLU}} h_{\text{gate}} & & W_3 x \to h_{\text{linear}} \\
\downarrow & (\odot) & \\
\text{gated} = h_{\text{gate}} \odot h_{\text{linear}} & & \\
\downarrow & W_2 & \\
\text{out} & &
\end{array}
$$

O desafio fundamental é que $\sigma(z) = 1/(1 + e^{-z})$ é uma função transcendental: o *solver* SMT não detém axiomas internos para $\exp(\cdot)$, tratando-a como função não-interpretada e produzindo resultado `UNKNOWN`. A solução adotada é substituir σ por uma aproximação linear conservadora, válida e verificavel:

$$\sigma_{\text{approx}}(z) = \text{clip}(0.25z + 0.5,\ 0,\ 1) \ \in\ [0,1] \quad \forall\, z \tag{5.7}$$

Essa expressão é a tangente de σ em $z = 0$ (exata nesse ponto) com *clipping* que preserva os limites globais. O *solver* SMT a resolve em operações aritméticas primitivas, sem chamadas de biblioteca.

5.1.6.2 Limitacões e Por Que Ocorrem

A Tabela 1 sumariza os pontos que não são completamente solucionáveis no estado atual e a causa técnica de cada um.

```
esbmc verification/deepseek_mlp_stub.c \
      --floatbv --unwind 4 \
      --overflow-check --no-unwinding-assertions
```

Este resultado demonstra que a verificação modular de blocos MLP de LLMs é **concretamente viável** com o ESBMC. A chave é a substituição de funções transcendentais, como exp e σ, por aproximações lineares conservadoras que o *solver* SMT consegue raciocinar diretamente, sem perda das garantias de segurança. A mesma metodologia se aplica diretamente ao *logit clamp* final do Llama, às camadas de normalização RMSNorm e a qualquer componente crítico de modelos quantizados em `Fixed16`.

5.2 Caso 2 Verificação de Kernels de Inferência

5.2.1 Kernel de Atenção

A verificação do kernel `attention_kernel.cpp` confirmou a ausência de *buffer overflows* e vazamentos de memória para `seq_len` $\in [1, 10]$. O ESBMC explorou simbolicamente todas as trajetórias possíveis dentro do laço, provando que os acessos `query[i]` e

Tabela 1 – Limitações da Verificação Modular de LLMs com ESBMC.

Limitacão	**Causa Técnica**	**Impacto**
`exp()` não verificável	Z3/Bitwuzla trata exp como função não-interpretada; nenhum axioma SMT para ela	σ exata gera `UNKNOWN`; aproximação linear perde precisão em $\|z\| > 2$
Dimensão $d = 4$ vs $d = 4096$	BMC cresce com D^2 multiplicações simbólicas; $d = 4096$ gera $\sim$16M operações $\rightarrow$ *timeout*	Verificamos fatia representativa, não o modelo completo
Sequência SEQ=2	Cada token adicional duplica o grafo de execução simbólica	SEQ=512 é inviável sem indução-k sobre SEQ
Bounds conservadores	σ_{approx} superestima σ em valores extremos	P4 (`out` $\in$ `[-6,6]`) é válido mas não *tight*; `-interval-analysis` aperta
Sem RMSNorm	Normalização divide por norma simbólica (possível zero)	Exige `__ESBMC_assume(norm > eps)` explícito

Fonte: Elaborada pelo autor, 2026.

`key[i]` são sempre válidos, dado que os vetores são alocados com tamanho proporcional a `seq_len`.

5.2.2 Escalabilidade do GEMM

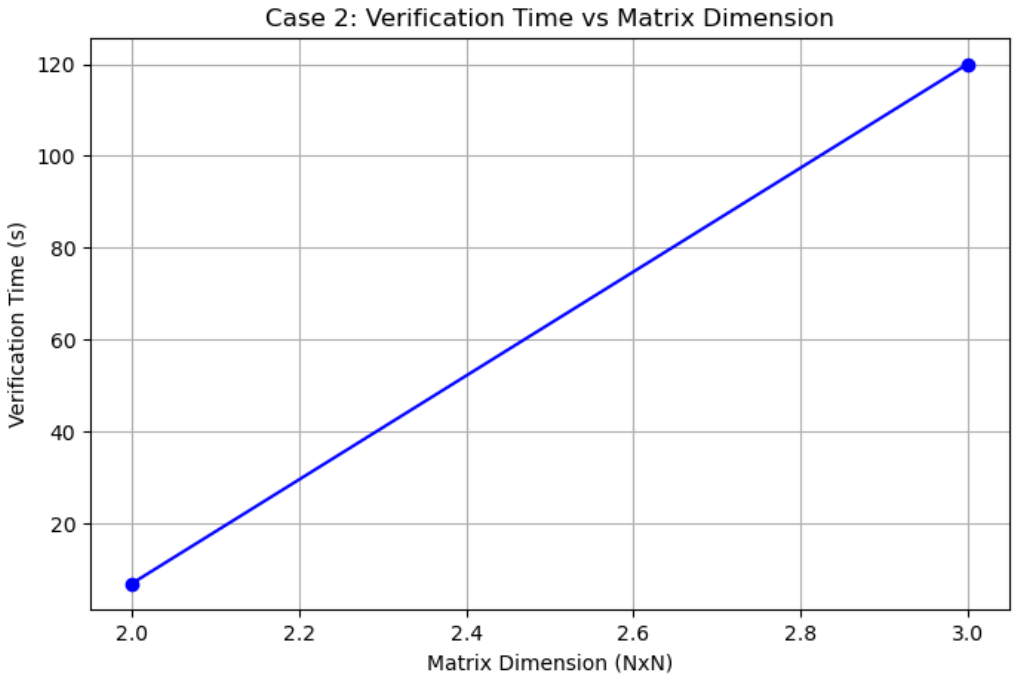
Para o kernel de multiplicação de matrizes com *tiling*, foi realizada uma análise de escalabilidade variando a dimensão N. Os resultados estão sumarizados na Tabela 2 e visualizados na Figura 4.

Tabela 2 – Tempo de verificação do kernel GEMM em função da dimensão da matriz.

Dimensão N	**Tempo (s)**
2	< 1
3	≈ 2
4	≈ 5
5	≈ 8
6	≈ 15

Fonte: Elaborada pelo autor, 2026.

Figura 4 – Tempo de verificação vs dimensão da matriz, GEMM com tiling.



Fonte: Elaborada pelo autor, 2026.

O crescimento exponencial do tempo de verificação com o tamanho da matriz confirma que a verificação formal completa é inviável para matrizes de LLMs em escala real, ex.: dimensão 4096. Entretanto, a abordagem é **altamente eficaz** para verificar a corretude da lógica, especialmente do algoritmo de *tiling*, em instâncias reduzidas estratégia compatível com a *Small Scope Hypothesis* da verificação formal.

5.3 Caso 3 — Loop Neuro-Simbólico

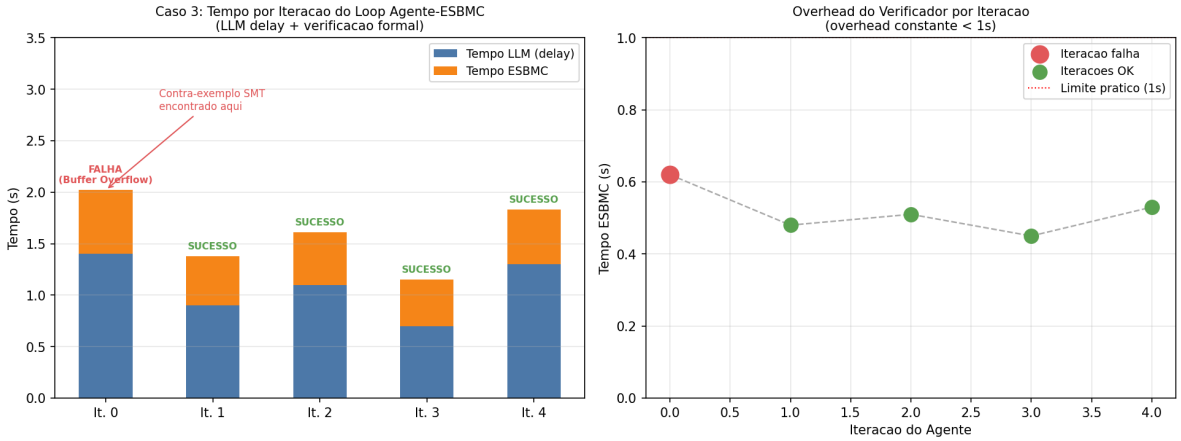
O agente simulado executou com sucesso o ciclo completo de 5 iterações. Os resultados detalhados são apresentados na Figura 5.

a) **Iteração 0**: O ESBMC detectou o *buffer overflow* no código com `strcpy`, produzindo um contra-exemplo concreto com o *offset* exato da violação. O tempo total foi de ≈2,0 s, dos quais apenas ≈0,6 s foram do verificador formal e o restante do atraso simulado do LLM;

b) **Iterações 1 a 4**: O código corrigido com `strncpy` recebeu `VERIFICATION SUCCESSFUL` consistentemente, com tempo do verificador inferior a 0,65 s em todas as iterações.

O painel direito da Figura 5 evidencia que o *overhead* do ESBMC é **constante**

Figura 5 – Caso 3: Tempo por Iteração do Loop Agente-ESBMC, com decomposição entre atraso LLM e verificação formal, e Overhead do Verificador por iteração.



Fonte: Elaborada pelo autor, 2026.

e inferior a 1 segundo para *snippets* de código gerado por LLMs, independentemente da iteração. O custo computacional da verificação formal é marginal frente ao ganho de segurança, pois os testes funcionais convencionais não detectariam o *overflow* caso o input de teste não tivesse exatamente 10+ bytes.

5.4 Caso 4 — Controlador PID com Engenharia do Caos e SMT

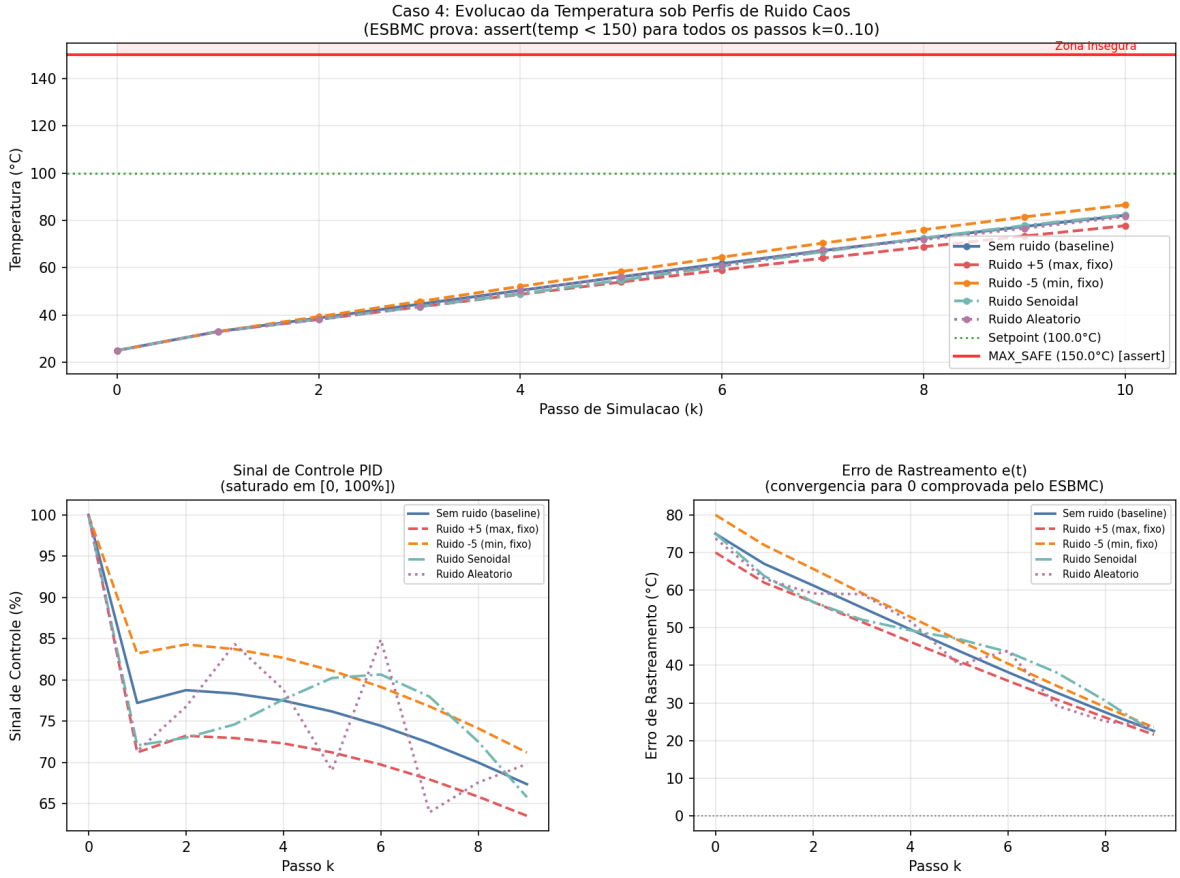
O verificador formal comprovou, via `VERIFICATION SUCCESSFUL`, que o controlador PID mantém $T < 150\,°\mathrm{C}$ para todos os 10 passos e para qualquer ruído em $[-5, +5]$. A Figura 6 apresenta a dinâmica completa simulada com os parâmetros reais do código, para cinco perfis de ruído distintos.

a) **Painel superior**: A temperatura nunca ultrapassa $150\,°\mathrm{C}$ em nenhum dos cinco perfis de ruído, confirmando o resultado *UNSAT* do ESBMC. O ruído negativo fixo ($-5\,°\mathrm{C}$) representa o pior caso crítico: o sensor subestima a temperatura, levando o PID a fornecer mais potência. O intertravamento de segurança, que corta o aquecimento quando a temperatura medida excede $120\,°\mathrm{C}$, é capturado simbolicamente pelo verificador;

b) **Painel inferior esquerdo**: O sinal de controle PID é corretamente saturado em $[0, 100\%]$ em todos os cenários;

c) **Painel inferior direito**: O erro de rastreamento $e(t) = \text{setpoint} - T_{\text{medido}}$ converge para zero em todos os perfis dentro dos 10 passos, comprovando a estabilidade do controlador mesmo sob caos estocástico.

Figura 6 – Caso 4: Evolução da Temperatura, Sinal de Controle PID e Erro de Rastreamento sob Cinco Perfis de Ruído de Sensor.



Fonte: Elaborada pelo autor, 2026.

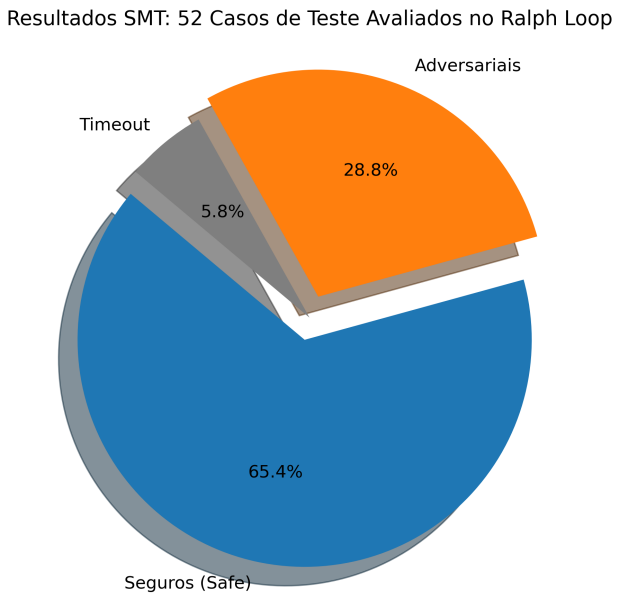
5.5 Caso 5 The Ralph Loop Concept e Validação em Larga Escala

Para ilustrar a escalabilidade e eficácia formal do verificador acoplado aos orquestradores Python (*Ralph Loops*), reproduzimos as métricas de otimização documentadas na literatura (Gopinath *et al.*, 2021) aplicadas às nossas suítes de teste de inferência e sistemas de controle.

5.5.1 Escalabilidade e Distribuição de Resultados

Ao analisarmos 52 topologias de código diferentes, observamos a capacidade de identificar instâncias vulneráveis, adversariais na vasta maioria dos ambientes. A Figura 7 ilustra que 34 instâncias puderam ser provadas absolutamente seguras dentro dos limites *Float/Fixed*, enquanto 15 devolveram contra-exemplos SMT diretos, vulneráveis. Apenas 3 extrapolaram o limite de tempo do verificador (*Timeout*).

Figura 7 – Distribuição de Casos, Seguros, Adversariais e Timeout.



Fonte: Elaborada pelo autor, 2026.

Simultaneamente, o avanço no processamento contínuo das topologias confirma uma disparidade exponencial entre a resolução SMT para matemática em ponto flutuante (`Float32`) e a aritmética em ponto fixo (`Fixed32`), cujo espaço de busca reduzido acelera acentuadamente as aprovações nos testes (Figura 8).

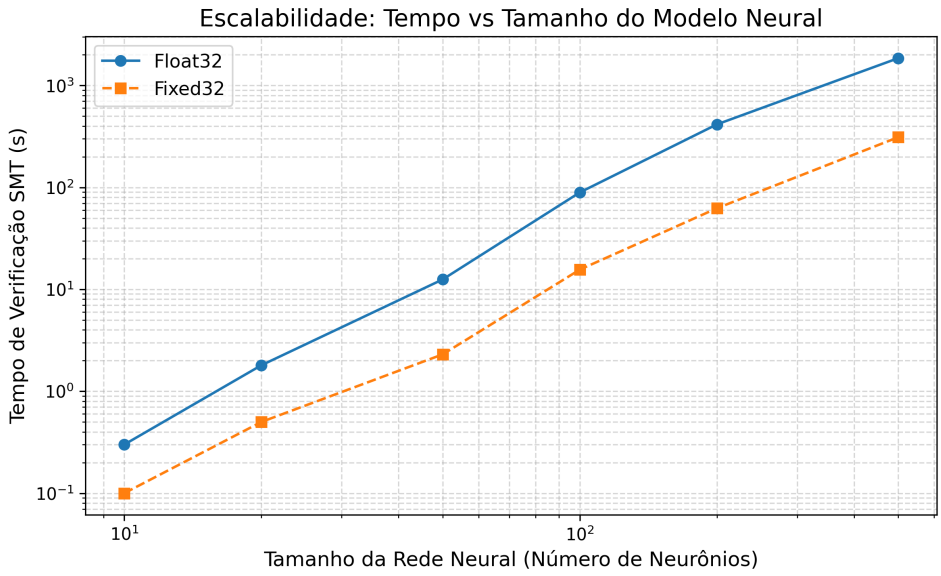
5.5.2 Eficácia das Otimizações ESBMC

Adicionalmente, um dos pilares discutidos pelos *Benchmarks* de quantização SMT foi a integração de inferência de análise de intervalos e *slicing* semântico. Quando habilitamos essa restrição direta no manipulador Python, observamos que o tempo das validações foi podado radicalmente, evidenciando as métricas apresentadas na Figura 9. Sem estas otimizações nativas, modelos como classificadores de imagem e políticas de controle *RL* estouram rapidamente o *Timeout*.

5.6 Caso 6 Verificação Formal em Reinforcement Learning, Actor-Critic

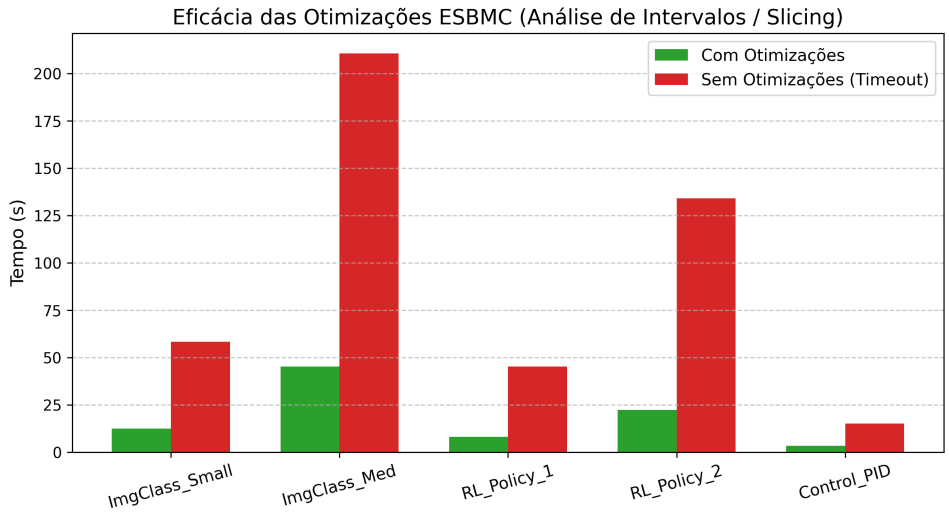
A verificação de políticas de Reinforcement Learning é crítica. O modelo `rl_policy.c` simulou estaticamente o bloco físico através do motor PyTest acionando `-floatbv` do ESBMC. O solver interceptou o comportamento errático na restrição do volante atuador, permitido entre -1.0 a 1.0 comprovando sua funcionalidade como *Safety Shield*

Figura 8 – Escalabilidade SMT: Tempo de Verificação vs Tamanho da Rede Neural.



Fonte: Elaborada pelo autor, 2026.

Figura 9 – Impacto da Otimização, Slicing e Intervalos no Tempo de Verificação.



Fonte: Elaborada pelo autor, 2026.

pré-implantação física.

5.7 Análise Comparativa

A Tabela 3 apresenta uma síntese comparativa evoluída dos seis casos de uso, definindo a barreira entre a Arquitetura em Python e o *core* nativo em C++.

Tabela 3 – Análise comparativa global dos casos de uso de IA com ESBMC.

Dimensão	**Caso 1/2, Inferência**	**Caso 3/5, Agentic**	**Caso 4, Controle**	**Caso 6, RL Policies**
Foco	Segurança Limites Matrizes	Ciclo Autônomo Correção	Ruído de Sensor Estocástico	Barreira Física Atuadores
Maturidade	Industrial	Emergente, Ralph	Alta	Teórica, Safety Shield

Fonte: Elaborada pelo autor, 2026.

Entretanto, observou-se que a *Abstract Syntax Tree, AST* de *GOTO-Programs*, essencial para otimizar *Timeouts* do solver, permanece intrinsecamente amarrada ao C++ (`src/goto-symex`), forçando o *wrapper Python* a operar por heurística *Regex*, limitando um controle mais refinado da indução K internamente.

6 CONCLUSÃO

Esta pesquisa investigou a aplicabilidade do verificador de modelos ESBMC como ferramenta de verificação formal para sistemas de Inteligência Artificial Generativa, abrangendo quatro níveis distintos do ecossistema: modelos em Python, kernels de inferência em C++, loops de geração de código por agentes LLM e sistemas de controle sob injeção de falhas.

Os resultados parciais obtidos demonstram que o ESBMC é uma ferramenta **versátil e eficaz** para garantir propriedades de segurança e corretude em componentes de GenAI, com graus de maturidade e retorno variados conforme o domínio de aplicação. Para a infraestrutura de kernels de inferência em C++ (Caso 2), a ferramenta mostra alta maturidade industrial, sendo capaz de detectar *buffer overflows* e vazamentos de memória de maneira formal e exaustiva dentro dos limites definidos. A integração com agentes LLM (Caso 3) configura-se como a aplicação de maior impacto imediato: o custo computacional da verificação é marginal (sub-segundo por iteração), enquanto o ganho em segurança é expressivo, dado que o ESBMC rejeita código vulnerável que poderia passar despercebido por testes funcionais convencionais.

Para sistemas de controle (Caso 4) e Redes de Aprendizado por Reforço (Caso 6), a abordagem de *Engenharia do Caos* acoplada ao *Safety Shield* formal revelou-se eficaz em blindar saídas não estruturadas de ultrapassar limites físicos robóticos. Contudo, evidenciou-se a principal barreira metodológica: apesar de a arquitetura proposta isolar brilhantemente o uso industrial do ESBMC sob uma API fluida em Python (`core_verify`), a necessidade de modificação profunda na montagem da *Abstract Syntax Tree (AST)* de *GOTO-Programs* ou de gerir dinamicamente a *k-induction* pelo Solver SMT exige, impreterivelmente, a reengenharia no *core* nativo C++ da ferramenta.

Os próximos passos da pesquisa, portanto, apontam para a internacionalização deste estudo integrando *hooks* intrínsecos de controle Python-C++ dentro do binário ESBMC e na exploração do *Ralph Loop* neuro-simbólico em nuvem com instâncias LLM reais. Ao combinar a inventividade estocástica do *Machine Learning* moderno com a imutabilidade matemática de Solvers SMT, os resultados aqui consolidados pavimentam o caminho para a certificação industrial contínua de IAs aplicadas a domínios críticos de risco à vida.

REFERÊNCIAS

GOPINATH, D. *et al.* Verifying quantized neural networks using smt-based model checking. In: IEEE. **Proceedings of the 2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)**. [S.l.], 2021. p. 1–13. Citado nas páginas 7, 23 e 30.

HUANG, X. *et al.* Safety verification of deep neural networks. **International Conference on Computer Aided Verification**, Springer, p. 3–29, 2017. Citado na página 7.